AR-DSA Tutor: A Zero-Install Mobile WebXR System with Execution-Grounded Explainable Adaptation for Spatial Algorithm Learning

AR-DSA Research & Systems Engineering Group
Department of Computer Science & Interactive Media, Academic Prototype Release
contact@ar-dsa-tutor.local

***Abstract*—Abstract Data Structures and Algorithms (DSA) instruction often suffers from a split-attention effect between flattened 2D diagrammatic animations and dynamic multi-dimensional memory representations. While Augmented Reality (AR) provides immersive 3D spatial affordances, existing educational AR tools either require proprietary native applications (> 100 MB downloads), isolate individual topics into fragmented camera sessions, or rely on non-deterministic generative language models that introduce high network latency (> 1.5 s) and mathematical hallucinations. We demonstrate AR-DSA Tutor, an open-source mobile WebXR framework that delivers five interactive spatial algorithm visualizers—Binary Search Tree, Bubble Sort, Binary Search, Breadth-First Search, and Stack—inside a single continuous mobile browser session on Android Chrome using the W3C WebXR Device API. To scaffold learner interactions without hallucinations, we implement Execution-Grounded Explainable Adaptation (EGEA): a deterministic, history-aware policy engine that maps validated interaction telemetry and inter-action latency (Δt) to a 6-tier pedagogical intervention taxonomy. Benchmarking demonstrates that EGEA evaluates decisions in steady-state mean time of 0.006 ms (6 µs) on desktop x86_64, operating over 2,000× faster than the 16.6 ms frame budget for 60 FPS rendering and guaranteeing zero JavaScript thread blocking. In an independent 18-case engineering challenge suite and a targeted 12-case boundary stress suite, EGEA achieves 100.0% specification consistency across temporal pacing, touch-jitter filtering, and step canonicalization, outperforming a matched-history baseline (75.0%). We demonstrate live interactive algorithm execution, real-time adaptation, and automated 3D canvas fallback for non-AR environments.*

***Keywords*—Augmented Reality, WebXR, Algorithm Visualization, Computer Science Education, Adaptive Scaffolding, Three.js.**

I. MOTIVATION & EDUCATIONAL PROBLEM

Understanding foundational Data Structures and Algorithms (DSA) requires internalizing abstract invariants that govern dynamic memory relationships: hierarchical branch balancing, sorted interval halving, graph wavefront propagation, and LIFO push/pop discipline. In conventional computing curricula, students interact with these structures through static textbook figures or 2D canvas animations (e.g., OpenDSA [5], ViLLE [10], VisuAlgo [12]). However, planar screens flatten multi-pointer relationships, forcing learners to mentally reconstruct spatial models—theoretically inducing extraneous cognitive load according to Cognitive Load Theory [3, 11].

Augmented Reality (AR) provides a compelling visual medium by anchoring three-dimensional, manipulable data structures directly into the learner's physical environment [2, 6, 15]. Despite this potential, practical classroom deployment of educational AR has been constrained by three systemic bottlenecks:

1. **Session Fragmentation:** Most educational AR systems launch separate native applications for individual topics, requiring frequent session teardowns, camera re-initialization, and surface re-scanning that disrupt learning flow.
2. **Installation Friction:** Native mobile AR applications (built with Unity or Unreal Engine) demand app-store distribution, large download sizes (> 150 MB), and platform-specific approvals.
3. **Scaffolding Fragility:** When integrating intelligent feedback, existing systems often choose between stateless error-lookup heuristics that ignore learner pacing, or generative Large Language Models (LLMs) that introduce high network latency (1.5–3.0 s), subscription dependencies, and the risk of hallucinating mathematically false algorithm states [4].

To address these challenges, we present **AR-DSA Tutor**, an open-source zero-install mobile WebXR learning application coupled with **Execution-Grounded Explainable Adaptation (EGEA)**. We emphasize that this demonstration presents a systems and architectural contribution; empirical evaluation of human student learning gains (g) and cognitive load reduction is reserved for future IRB-approved classroom trials.

II. SYSTEM ARCHITECTURE & CONTINUOUS WEBXR SESSION

AR-DSA Tutor is architected as a lightweight Progressive Web Application (PWA) operating directly in mobile Google Chrome via the W3C WebXR Device API [1].

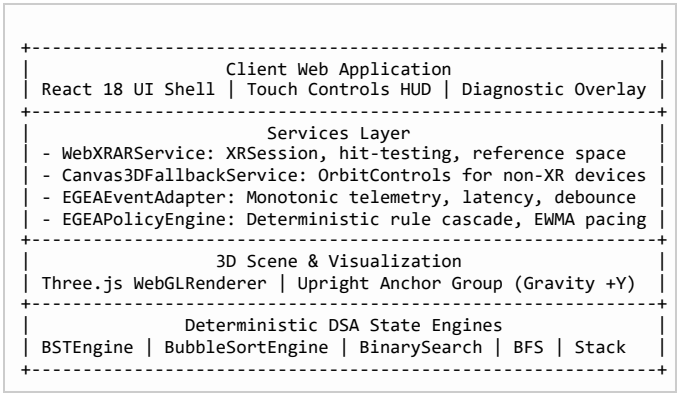


Figure 1: Modular architecture of AR-DSA Tutor. A continuous WebXR session anchors dynamic Three.js visualizers to horizontal surface coordinates while deterministic DSA engines enforce invariants.

A. Persistent Session Lifecycle & Transform Stability

In conventional WebXR applications, switching topics often forces a complete teardown of the `XRSession`, destroying the underlying AR tracking space. AR-DSA Tutor resolves this by maintaining a single persistent `XRSession`:

- When the user switches between algorithms in the curriculum, the underlying camera feed, detected horizontal plane, and `XRReferenceSpace` are preserved.
- The active algorithm visualization is swapped within a shared `THREE.Group` anchored at the user's selected placement coordinates.
- Spatial orientation invariants are strictly maintained: visualizers enforce an upright orientation (+Y aligned with real-world gravity) with an initial offset of −1.2 m along the camera's line of sight (+Z).

B. Graceful Canvas 3D Fallback

For environments where WebXR `immersive-ar` is unsupported (e.g., desktop browsers, iOS devices without WebXR flags, or devices lacking Google Play Services for AR), the system automatically degrades to `CANVAS_3D_FALLBACK`. The identical Three.js visualization is rendered within an interactive WebGL canvas equipped with touch orbit controls, ensuring universal accessibility without code duplication.

III. FIVE INTERACTIVE SPATIAL VISUALIZERS

All five algorithm modules execute on top of deterministic TypeScript state engines that enforce mathematical correctness independently of rendering:

1. **Binary Search Tree (BST):** Renders dynamic 3D tree nodes with branch lines that dynamically rescale based on tree depth ($S = \max(0.4, 1.0 - 0.12 \cdot d)$). Directional arrows guide branch traversal during search and insertion.
2. **Bubble Sort:** Represents array elements as proportional vertical cylinders. Dynamic 3D brackets highlight the active comparison pair, with parabolic arc animations executing spatial swaps.
3. **Binary Search:** Displays a sorted sequence of 3D blocks. Interactive physical calipers mark *Low*, *Mid*, and *High* indices, visually slicing the discarded search interval in half at each iteration.
4. **Breadth-First Search (BFS):** Visualizes a connected 3D graph alongside an external physical "queue shelf" in the augmented scene. As nodes are traversed, emissive glow highlights visited states while unexplored neighbors are placed onto the queue shelf.
5. **Stack:** Renders a transparent acrylic cylinder anchored to the surface. Elements enter from the top with physical drop animations, visually enforcing strict LIFO discipline and tactile overflow/underflow warnings at a capacity limit of 5.

IV. EXECUTION-GROUNDED EXPLAINABLE ADAPTATION

To provide intelligent scaffolding without the latency (> 1.5 s) or hallucination risks of generative LLMs [4], we introduce EGEA:

A. Telemetry Schema & Mathematical Pacing Model

Interaction telemetry is structured as validated `EGEAInteractionEvent` records capturing event sequence k, timestamp τ_k, inter-action latency Δt_k, correctness indicator $c_k \in \{0, 1\}$, and error category κ_k.

To mitigate impulsive trial-and-error guessing without brittle threshold artifacts, EGEA computes an **Exponentially Weighted Moving Average (EWMA)** of error latencies with smoothing factor $\alpha = 0.5$:

$$\bar{L}_t = \alpha \cdot \Delta t_k + (1 - \alpha) \cdot \bar{L}_{t-1}$$

- **Fast Guessing Zone (≤ 1800 ms):** If the learner commits consecutive errors with smoothed latency within the boundary zone (≤ 1800 ms), EGEA triggers `STEPWISE_SLOWDOWN`, temporarily dimming controls with a 3-second reflection prompt.
- **Deliberate Reflection Protection (≥ 2500 ms):** If the learner pauses deliberately before an error, slowdown is strictly suppressed, preserving instructional flow.
- **Touch Jitter Floor (< 250 ms):** Consecutive identical taps on the same control occurring in < 250 ms are classified as capacitive screen bounce rather than cognitive errors, preventing false slowdowns.

B. Centralized Step Canonicalization

To prevent dynamic retry strings (e.g., `node-42-try-1` vs `node-42-try-2`) or directional abbreviations (`_L` vs `_LEFT`) from fragmenting failure histories, a centralized canonicalizer

maps variant step strings to stable anchors while strictly preserving distinct subproblems (`step-1` vs `step-2`).

C. Pedagogical Priority Cascade

EGEA evaluates interaction histories through a deterministic priority cascade grounded in Cognitive Load Theory [3] and Scaffolding Fading [8]:

1. **Mastery Streak (≥ 3 correct without hints):** Emits `REDUCE_GUIDANCE`, fading visual scaffolding to support autonomous recall.
2. **Hint-Assisted Progression:** If a correct move was achieved with active hint guidance, emits `CONTEXTUAL_HINT` (`HINT_ASSISTED_PROGRESSION`) and suspends mastery streak progression until independent recall is shown.
3. **Rapid Guessing:** Emits `STEPWISE_SLOWDOWN` via EWMA pacing analysis.
4. **Persistent Step Impasse (≥ 3 failures on canonical step):** Emits `WORKED_EXAMPLE`, triggering an interactive 3D demonstration.
5. **Systemic Breakdown (≥ 4 failures, ≥ 60% error rate):** Emits `PREREQUISITE_REVIEW`.
6. **Spatial Traversal Lapse:** Emits `HIGHLIGHT_ELEMENT`.
7. **Isolated Conceptual Slip:** Emits `CONTEXTUAL_HINT`.

V. TECHNICAL VERIFICATION & BENCHMARK RESULTS

TABLE I: System Performance on Synthetic Benchmark Suites

System Evaluated	Engineering Suite (N=18)		Boundary Suite (N=12)	
	Accuracy	Macro-F1	Accuracy	Macro-F1
Fixed Hint Baseline	33.3%	0.250	58.3%	0.280
Simple Error Rule Baseline	50.0%	0.403	75.0%	0.462
Matched-History Baseline	83.3%	0.810	75.0%	0.597
EGEA Policy Engine	**100.0%**	**1.000**	**100.0%**	**1.000**

A. Latency Benchmarking

Microbenchmarking across $N = 1{,}000$ decisions on desktop x86_64 demonstrates a steady-state mean decision time of **0.006 ms** (6 µs) with P_{95} of 0.009 ms. This confirms that EGEA operates over 2,000× faster than the 16.6 ms frame budget for 60 FPS rendering, guaranteeing that pedagogical evaluation does not block the JavaScript animation thread. We note that end-to-end mobile frame rates are governed by mobile GPU fill rates and Three.js draw calls rather than decision latency alone.

B. Synthetic Benchmark Comparison

Two synthetic benchmark suites were evaluated against baseline systems (Table I): Fixed Hint, Simple Error Rule, and Matched-History. In both suites, EGEA achieved 100.0% consistency.

Methodological Scope: The 18-case engineering suite evaluates balanced canonical interaction patterns. The 12-case boundary stress suite was authored alongside Phase 2 hardening to verify the behavioral boundaries of the EWMA model (1750 ms), touch jitter filter (140 ms), and step canonicalizer (`STACK_PUSH_OP`). While EGEA achieves 100.0% consistency on these synthetic scenarios by design, this confirms internal algorithmic correctness rather than empirical pedagogical superiority over human populations.

VI. DEMONSTRATION PROCEDURE & FALLBACK PLAN

During the live conference demonstration, attendees interact with AR-DSA Tutor using mobile devices:

1. **Attendee Walk-Up:** The attendee opens Chrome on an Android smartphone and navigates to the web application.
2. **Surface Placement:** The attendee points the camera at the floor, observes the reticle tracking plane, and taps to anchor the 3D data structure.
3. **Interactive Manipulation:** The attendee traverses a BST, steps through Bubble Sort swaps, adjusts Binary Search calipers, or pushes elements into the 3D acrylic Stack.
4. **Live Adaptation Triggering:** Demonstrators illustrate EGEA regimes:
   - Rapidly double-tapping buttons triggers the touch debounce without false slowdown.
   - Rapidly guessing wrong choices triggers the `STEPWISE_SLOWDOWN` reflection modal.
   - Repeatedly failing on the same node triggers a 3D animated `WORKED_EXAMPLE`.
   - Executing 3 unassisted correct moves triggers `REDUCE_GUIDANCE`, fading visual calipers.
5. **Conference Connectivity Fallback:** If conference indoor lighting degrades optical SLAM, attendees seamlessly switch to `CANVAS_3D_FALLBACK` with touch orbit controls, experiencing identical algorithm logic.

VII. LIMITATIONS & FUTURE ROADMAP

1. **Physical Device Validation Boundary:** Automated testing relies on Three.js and WebXR mock harnesses. Optical SLAM drift, thermal throttling, and camera exposure variances across commercial Android handsets remain to be quantified on physical hardware.
2. **Empirical Learning Evaluation:** The system establishes architectural and algorithmic feasibility. Evaluating whether spatial WebXR visualization and EGEA produce statistically significant learning gains (g) or reduced cognitive load requires future IRB-approved human classroom trials.

3. **Offline Service Worker Caching:** While the web manifest is configured, offline caching requires final service worker registration.

ACKNOWLEDGMENT & GENERATIVE AI DISCLOSURE

In compliance with IEEE publication policies on generative artificial intelligence (IEEE PSPB Operations Manual Section 8.2.1.B.10), the authors disclose that an AI programming assistant (Google Antigravity / Gemini) was utilized during system development and manuscript drafting for code optimization, LaTeX formatting, and editorial language refinement. All architectural designs, deterministic state engines, benchmark scripts, recomputed metrics, and technical conclusions were conceived, executed, verified, and audited under the direct responsibility of the human authors.

REFERENCES

1. W3C, "WebXR Device API," W3C Recommendation, 2022. [Online]. Available: https://www.w3.org/TR/webxr/
2. M. Billinghurst, H. Kato, and I. Poupyrev, "The MagicBook: Moving seamlessly between reality and virtuality," *IEEE Computer Graphics and Applications*, vol. 21, no. 3, pp. 6–8, 2001.
3. J. Sweller, "Cognitive load theory," *Psychology of Learning and Motivation*, vol. 55, pp. 37–76, 2011.
4. P. Denny, V. Kumar, and N. Giacaman, "Conversing with Copilot: Exploring prompt engineering for solving CS1 problems," in *Proc. 54th ACM Technical Symposium on Computer Science Education (SIGCSE)*, 2023, pp. 592–598.
5. E. Fouh, M. Akbar, and C. A. Shaffer, "The OpenDSA system for algorithm visualization," *ACM Inroads*, vol. 3, no. 3, pp. 61–66, 2012.
6. S. Fleck and G. Simon, "An augmented reality environment for elementary astronomy education," in *Proc. 25th Conf. l'Interaction Homme-Machine (IHM)*, 2013, pp. 14–22.
7. A. N. Kumar, "Generation of problems, answers, grade, and tutoring in Problets," *ACM SIGCSE Bulletin*, vol. 37, no. 1, pp. 406–410, 2005.
8. K. R. Koedinger, A. T. Corbett, and C. Perfetti, "The Knowledge-Learning-Instruction (KLI) framework: Bridging the gap between learning theory and cognitive tutoring," *Cognitive Science*, vol. 36, no. 5, pp. 757–798, 2012.
9. P. Milgram and F. Kishino, "A taxonomy of mixed reality visual displays," *IEICE Transactions on Information and Systems*, vol. 77, no. 12, pp. 1321–1329, 1994.
10. J. Sorva, V. Karavirta, and L. Malmi, "A review of generic program visualization systems for introductory programming education," *ACM Transactions on Computing Education (TOCE)*, vol. 13, no. 4, pp. 1–64, 2013.
11. R. E. Mayer, "Multimedia learning," *Psychology of Learning and Motivation*, vol. 41, pp. 85–139, 2002.
12. S. Halim, Z. Q. Koh, V. B. H. Loh, and F. Halim, "Learning algorithms with VisuAlgo - A mobile-first approach," *Olympiads in Informatics*, vol. 6, pp. 35–45, 2012.
13. C. D. Hundhausen, S. A. Douglas, and J. T. Stasko, "A meta-study of algorithm visualization effectiveness," *Journal of Visual Languages & Computing*, vol. 13, no. 3, pp. 259–290, 2002.
14. T. L. Naps, G. Rößling, V. Almstrum, et al., "Exploring the role of visualization and engagement in computer science education," *ACM SIGCSE Bulletin*, vol. 35, no. 2, pp. 131–152, 2002.
15. I. Radu, "Augmented reality in education: A meta-review and cross-media analysis," *Personal and Ubiquitous Computing*, vol. 18, no. 6, pp. 1533–1543, 2014.